REPUBLIC OF THE PHILIPPINES
TECHNOLOGICAL UNIVERSITY OF THE PHILIPPINES
TAGUIG CITY
Km. 14 East Service Road, Western Bicutan, Taguig City 1630
www.tupt.edu.ph | Tel. No.: (632) 8247-5250 to 54

The Technological University of the Philippines shall be a premier state university with recognized excellence in engineering and technology education at par with the leading universities in the ASEAN region.

HUMAN RESOURCE MANAGEMENT SYSTEM

A Java Swing/AWT Desktop Application Using JDBC and Apache Derby

PROJECT DOCUMENTATION

Submitted in partial fulfillment of the requirements for the course in Information Technology

SUBMITTED BY

Rebanal, Ron Harry
Calaramo, Carl
Pablo, John Clarence
BSIT-T-2B-T

SUBMITTED TO

Ms. Maria Cristina D. Baloloy

2026

ABSTRACT

Organizations that maintain employee information in paper files and disconnected spreadsheets can experience slow record retrieval, duplicate encoding, inconsistent employee details, and time-consuming attendance and leave monitoring. This project presents a Human Resource Management System (HRMS) desktop application designed to centralize employee, department, position, attendance, leave-request, and user-account records. The application is implemented in Java, with Swing and AWT providing the graphical interface, JDBC providing database connectivity, and Apache Derby operating as the embedded relational database. The system includes administrator and HR Staff roles, record-management forms, keyword search, input validation, confirmation dialogs, a dashboard, and read-only reports that can be exported to CSV. The database design uses primary keys, foreign keys, unique constraints, and check constraints to maintain relationships and prevent duplicate attendance entries for the same employee and date.

The application is organized into interface, data-access, model, and utility packages. Reusable DAO logic executes parameterized SQL statements, while the interface provides separate screens for the major HR modules. Passwords are stored as salted PBKDF2-HMAC-SHA256 hashes. The scope is intentionally limited to local desktop HR administration and excludes payroll, tax computation, recruitment, performance evaluation, biometric devices, and web-based employee self-service. Java source compilation and a password-hashing smoke test were completed during preparation of this package. Runtime tests against the embedded Derby engine, complete functional testing, and user-acceptance evaluation remain to be performed in NetBeans; this document therefore does not claim that those tests have passed.

Keywords: Human Resource Management System, Java Swing, Java AWT, JDBC, Apache Derby, employee records, attendance, leave management, CRUD, reporting

TABLE OF CONTENTS

Abstract	2
Keywords	2
Chapter 1 - Introduction	4
1.1 Background of the Study	4
1.2 System Overview	4
1.3 Problem Statement	4
1.4 Objectives	4
1.5 Scope and Limitations	5
1.6 Significance of the Project	5
1.7 Definition of Terms	5
1.8 System Requirements	6
Chapter 2 - System Analysis and Design	7
2.1 Existing System	7
2.2 Proposed System	7
2.3 Functional Requirements	8
2.4 Non-Functional Requirements	8
2.5 System Users	9
2.6 Use Case Diagram	9
2.7 System Flowchart	11
2.8 Data Flow Diagram	12
2.9 Entity Relationship Diagram	13
2.10 Database Design	14
Chapter 3 - System Development	15
3.1 Development Tools	15
3.2 Java Swing	15
3.3 Java AWT	15
3.4 JDBC	15
3.5 Apache Derby	15
3.6 System Architecture	16
3.7 Program Structure	16
3.8 Database Connection	17
3.9 CRUD Implementation	17
3.10 GUI and Reports	17
Chapter 4 - Testing	18
4.1 Test Plan	18
4.2 Test Cases	18
4.3 Security and User Acceptance Testing	19
Chapter 5 - Conclusion	20
5.1 Summary	20
5.2 Conclusions	20
5.3 Recommendations	20
5.4 Possible Future Improvements	20
References	21
Appendix A - Running the Project	22
Appendix B - Class Diagram	23
Appendix C - Table Relationship Diagram	24
Appendix D - Database Dictionary	25
Appendix E - SQL Statement Examples	27
Appendix F - System Interface Screenshots	28

CHAPTER 1 - INTRODUCTION

1.1 Background of the Study

Organizations depend on accurate employee information to assign responsibilities, monitor attendance, manage leave, and support administrative decisions. In a small office, paper folders and spreadsheets may initially seem adequate; however, repeated encoding, inconsistent copies, manual counting, and slow searching become more difficult as records increase. Human resource management includes maintaining the information needed to administer the employment relationship, and reliable records form a foundation for those activities (Armstrong & Taylor, 2020).

Electronic human resource management (e-HRM) applies information technology to HR processes and can reduce routine transactional work when the selected system is appropriate to the organization's needs (Bondarouk & Ruël, 2009; Strohmeier, 2007). A small organization or academic prototype does not necessarily need the breadth or cost of a full enterprise HR suite. A focused desktop application can demonstrate the fundamentals of user interfaces, relational database design, validation, access control, and CRUD processing.

This project develops a desktop-based HRMS using Java, Swing/AWT, JDBC, and Apache Derby. Derby is selected because it is a Java-based relational database that can run in embedded mode inside the application; this is consistent with the assignment requirement and the local single-computer scope (Apache Software Foundation, 2022).

1.2 System Overview

The HRMS manages employee profiles, departments, positions, daily attendance, leave requests, and system user accounts. After authentication, a user opens a dashboard with navigation to the authorized modules. Each operational module provides a form, a table of saved records, keyword search, and actions for adding or updating records. Deletion requires Administrator access and a confirmation dialog. Reports display employee, department headcount, attendance, and leave information in read-only tables; the displayed results may be exported as CSV.

The embedded database is stored locally in the project working directory by default. The JDBC URL is equivalent to `jdbc:derby:hrmsdb;create=true`. The application creates missing tables during startup and seeds demonstration accounts and starter departments/positions when the corresponding records do not yet exist. No separate MySQL server or phpMyAdmin installation is used.

1.3 Problem Statement

The project addresses the fragmentation of employee information across paper folders, logbooks, and separate spreadsheets. The specific questions are:

- How can employee records, departments, positions, attendance, and leave requests be stored in one organized database?
- How can repeated encoding and manual counting be reduced through search, database-backed tables, and summary reports?
- How can database relationships prevent orphan records and duplicate daily attendance?
- How can authentication and role-based restrictions separate routine HR work from administrative actions?
- How can the implementation be assessed through functional, data-integrity, error-handling, and user-acceptance testing?

1.4 Objectives

General objective. To design and develop a Java desktop HRMS that centralizes selected HR records, supports controlled record management, and uses JDBC with Apache Derby as required by the course project.

Specific objectives:

1. Analyze the weaknesses of the paper-based and spreadsheet-based process.
2. Design related tables and diagrams for users, departments, positions, employees, attendance, and leave requests.
3. Implement a readable Swing/AWT interface with login, dashboard navigation, management forms, search, reports, and logout.
4. Implement prepared JDBC statements, input validation, error handling, and integrity constraints.
5. Restrict user-account management, record deletion, and leave approval to authorized roles.
6. Prepare a structured test plan and user-acceptance questionnaire, recording actual results only after execution.

1.5 Scope and Limitations

Scope. The system includes login, Administrator and HR Staff roles, employees, departments, positions, attendance, leave requests, user accounts, search, dashboard summaries, and read-only CSV-exportable reports. The database uses the embedded Apache Derby engine and the application uses a local desktop deployment model.

Limitations. The system does not calculate payroll, salaries, taxes, or deductions; manage recruitment or performance evaluation; read biometric/ID hardware; send email/SMS notifications; or provide employee web self-service. It is not designed for concurrent access from multiple computers. The demo credentials must be changed before real data are entered. The project has not been evaluated for production HR privacy, legal, or cybersecurity compliance. Runtime database tests and user-acceptance evaluation remain pending.

1.6 Significance of the Project

HR Staff. Can record and retrieve employee, attendance, and leave data in one desktop workspace instead of searching separate files.

Management. Can review structured employee counts, department distribution, attendance, and pending leave information.

Employees. May benefit indirectly from more organized employee records and leave tracking.

Students and Developers. Can study the practical use of Java Swing/AWT, JDBC, relational design, constraints, CRUD, security controls, and testing.

Future Researchers. Can extend the prototype with leave balances, audit logs, stronger deployment controls, and additional reports.

1.7 Definition of Terms

Term	Definition in this project
HRMS	A desktop application used to organize employee records and selected HR administration tasks.
CRUD	Create, Read, Update, and Delete operations for database records.
Swing	Java GUI toolkit providing windows, panels, tables, buttons, and dialogs.
AWT	Java GUI and event framework used for layouts, fonts, colors, dimensions, and event handling.
JDBC	Java Database Connectivity, the standard Java API for connecting to and querying relational databases.
Apache Derby	A Java-based relational database with an embedded JDBC driver used by this application.
DAO	Data Access Object; a class responsible for executing database operations.

Term	Definition in this project
PBKDF2-HMAC-SHA256	A password-based key derivation method used to store salted password verifiers instead of plain-text passwords.
Primary key	A column or group of columns that uniquely identifies a record.
Foreign key	A constraint linking a column to a record in another table.
UAT	User Acceptance Testing, evaluation by intended users against defined tasks and criteria.

1.8 System Requirements

The following environment is required or recommended to build and run the academic desktop prototype. The application uses embedded Apache Derby and does not require a separate MySQL server, phpMyAdmin, or network database service.

Category	Requirement	Notes
Programming runtime	JDK 17 or newer	Use a Java Development Kit compatible with the Maven project configuration.
IDE / build	Apache NetBeans with Maven support	Open the folder containing pom.xml and allow dependency resolution.
Database	Apache Derby 10.16.1.1 (embedded JDBC)	Created and initialized locally by DatabaseSetup on first launch.
Operating system	Windows, Linux, or macOS with a compatible JDK	Portable at the Java level; this prototype has not been cross-platform tested.
Memory / storage	4 GB RAM recommended; writable local storage for dependencies and hrmsdb	This is a practical recommendation rather than a measured minimum.
Database administration	No external server or phpMyAdmin required	Back up the generated hrmsdb folder after closing the application.

CHAPTER 2 - SYSTEM ANALYSIS AND DESIGN

2.1 Existing System

The existing process described in the project is a manual or semi-manual HR workflow. Employee details are kept in paper folders and duplicated in spreadsheets. Attendance is written in logbooks, while leave requests are recorded on paper and filed separately. This setup depends on manual checking and does not update related copies automatically.

Area	Existing approach	Observed limitation
Employee records	Paper folders and separate contact spreadsheets	Searching takes time; duplicate details may disagree.
Department / position	Separate lists or spreadsheets	Assignments may be outdated after changes.
Attendance	Record attendance date, AM/PM time in/out, and status; an earlier Time Out is treated as next day; prevent duplicate employee/date rows.	Counting errors and slower summaries.
Leave	Paper request forms and separate absence list	Status and dates are harder to trace.
Reports	Manual counting and compilation	Repeated work and inconsistent totals.
Access	Physical file access	Confidential records may be seen by unauthorized persons.

2.2 Proposed System

The proposed system centralizes these records in a structured embedded relational database accessed by the Java desktop application. Users authenticate before entering the workspace. Data entry is checked before saving, user-provided values are bound through PreparedStatement objects, joined queries show department/position names instead of only identifiers, and database constraints protect record relationships. The application has a light interface with dark readable text, visible labels, standard buttons, searchable tables, and confirmation/error messages.

Module	Main functions
Authentication	Login validation, role-aware navigation, clear/reset, exit, sign out.
Dashboard	Employee totals, active staff, departments, pending leave, today attendance, recently added employees.
Employees	Create, view, update, search, and administrator-authorized deletion; link to department and position.
Departments / Positions	Maintain reference lists used by employee assignments.
Attendance	Record attendance date, AM/PM time in/out, and status; an earlier Time Out is treated as next day; prevent duplicate employee/date rows.
Leave Requests	File/edit request details; Administrator reviews and approves/rejects.
User Accounts	Administrator creates/updates/removes accounts and assigns roles.
Reports	Employee directory, department headcount, attendance register, leave register, pending leave; CSV export.

2.3 Functional Requirements

ID	Functional requirement
FR-01	Require a username and password and reject invalid credentials before showing the dashboard.
FR-02	Support Administrator and HR Staff roles with different access rights.
FR-03	Allow authorized users to add, view, update, search, and maintain employee records.
FR-04	Allow authorized users to add, view, update, search, and maintain departments.
FR-05	Allow authorized users to add, view, update, search, and maintain positions.
FR-06	Associate each employee with one valid department and one valid position.
FR-07	Record attendance date, time in/out using 12-hour AM/PM input, and status; treat an earlier Time Out as next-day clock-out.
FR-08	Prevent more than one attendance record for the same employee on the same date.
FR-09	File leave requests with employee, leave type, start/end dates, reason, status, and filing date.
FR-10	Restrict leave approval/rejection status changes to the Administrator.
FR-11	Search employee, attendance, leave, department, position, and user-account records by keyword where relevant.
FR-12	Validate required fields, date formats, valid AM/PM time values, email format, and leave-date ordering before saving.
FR-13	Confirm destructive operations; restrict record deletion to the Administrator and protect the active/last Administrator account.
FR-14	Allow the Administrator to create, update, and delete user accounts subject to safeguards.
FR-15	Provide login Clear/Reset, Exit, and sign-out controls.
FR-16	Display useful dashboard summaries from database queries.
FR-17	Display selected HR reports in a JTable and export displayed data to CSV.
FR-18	Store and retrieve all application records using JDBC and embedded Apache Derby.

2.4 Non-Functional Requirements

Category	Requirement
Usability	Use a consistent light theme, high-contrast text, clear labels, readable fonts, visible table headers, standard buttons, and organized navigation.

Category	Requirement
Performance	Complete common CRUD/search/report queries within a reasonable period for the intended academic prototype data size; measure during runtime testing.
Reliability	Persist successful changes in Derby and show actionable messages when database or input errors occur.
Security	Use JPasswordField, salted PBKDF2-HMAC-SHA256 password verification, prepared statements, and role-based controls.
Data integrity	Use primary keys, foreign keys, UNIQUE and CHECK constraints for unique identifiers, roles/status values, and date/time logic.
Portability	Run on a compatible Java 17+ environment with Maven resolving the embedded Derby dependency.
Maintainability	Separate UI, DAO, model, and utility classes; centralize connection/schema setup and reuse CRUD logic.
Error handling	Catch SQL and validation errors and present understandable messages without intentionally terminating the application.

2.5 System Users

User	Responsibilities and access
Administrator	Accesses all modules; manages user accounts; approves/rejects leave; performs record deletion; cannot delete the currently signed-in account or remove/demote the last Administrator.
HR Staff	Maintains employee, department, position, attendance, and leave request details; searches records and views/exports reports. Cannot manage user accounts, delete records, or alter leave approval status.

2.6 Use Case Diagram

The use case diagram summarizes the common tasks available to both user roles and identifies the Administrator-only operations. Authentication is required before a protected function can be accessed.

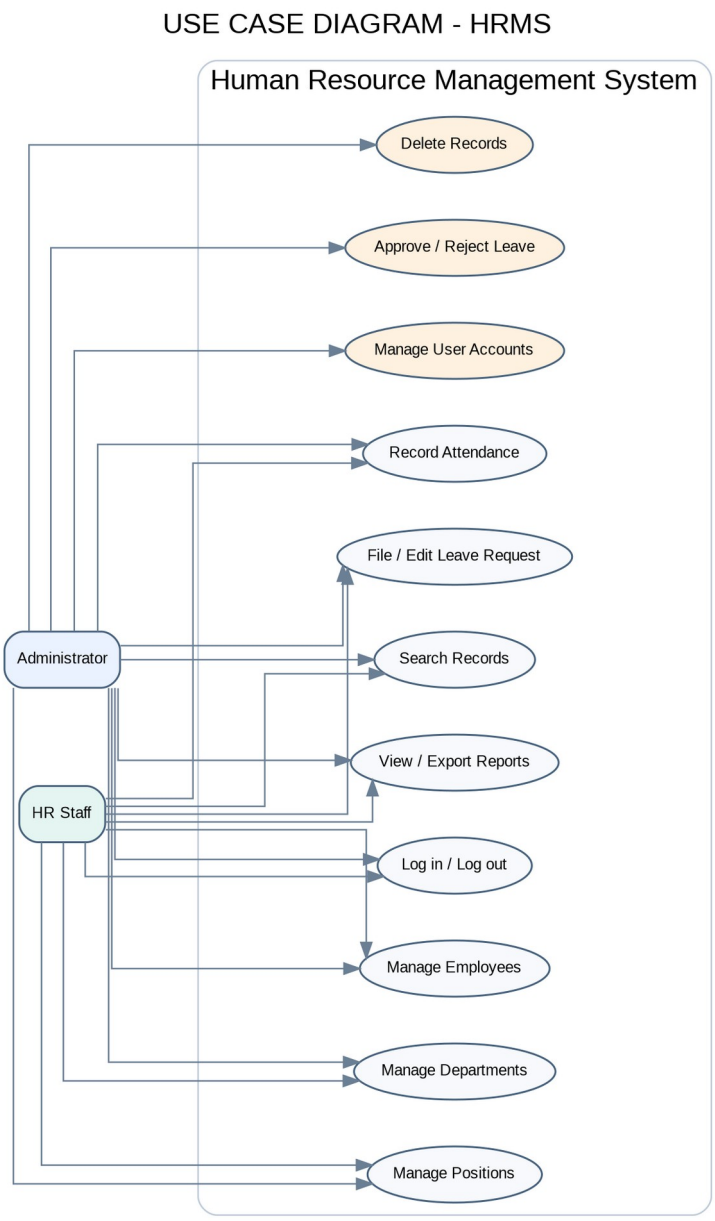


Figure 2.1. Use case model for the HRMS.

2.7 System Flowchart

On startup, the application initializes the local Derby database, creates missing tables, and removes the former same-day attendance-time constraint if it exists in an older database. After authentication, users select a module. The interface validates the input, submits a parameterized JDBC operation when valid, and displays the database result or an error message. For overnight attendance, Time Out earlier than Time In is interpreted as the following day. Logout returns the user to the login screen.

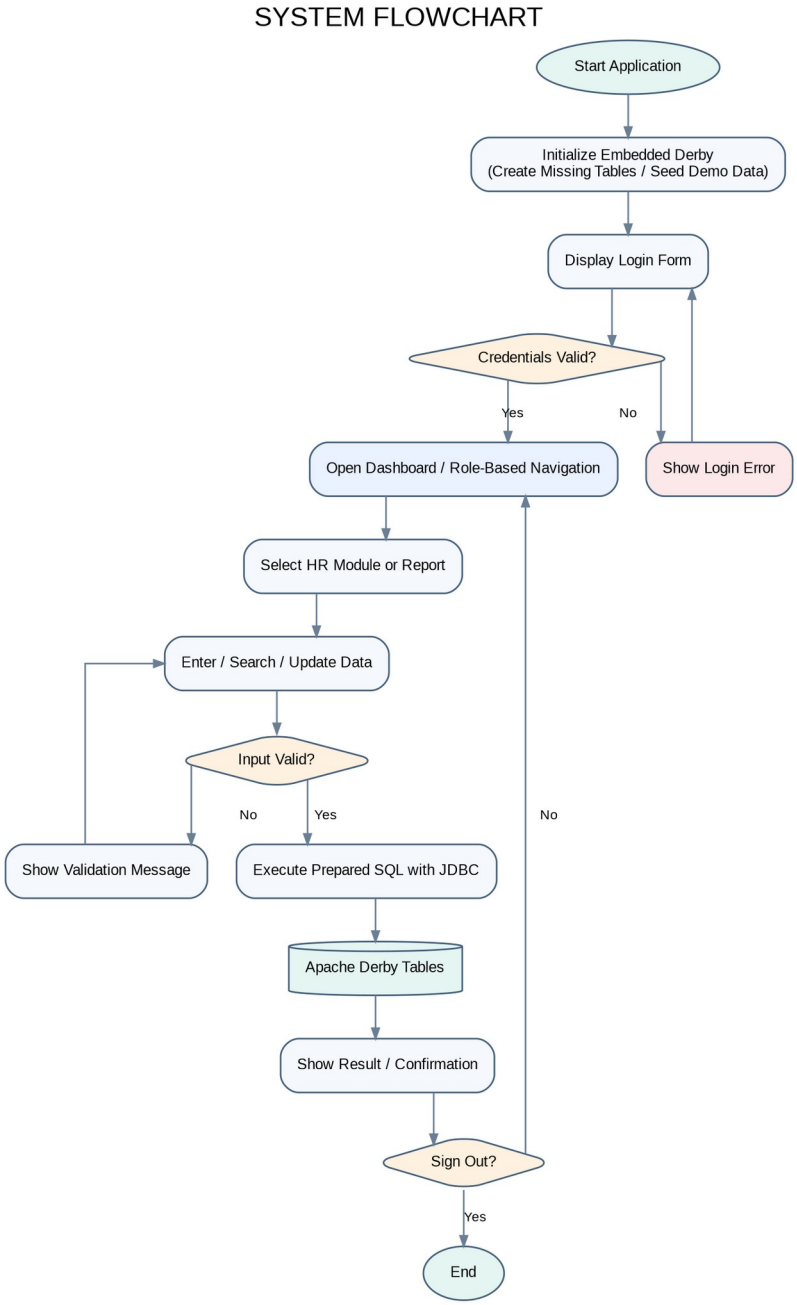


Figure 2.2. Application start-up, authentication, record processing, and logout flow.

2.8 Data Flow Diagram

The context-level data-flow model shows Administrators and HR Staff sending record-management, search, and report requests to the HRMS process. The application reads from and writes to the local Derby database and returns saved records, report rows, confirmations, or validation/database messages.

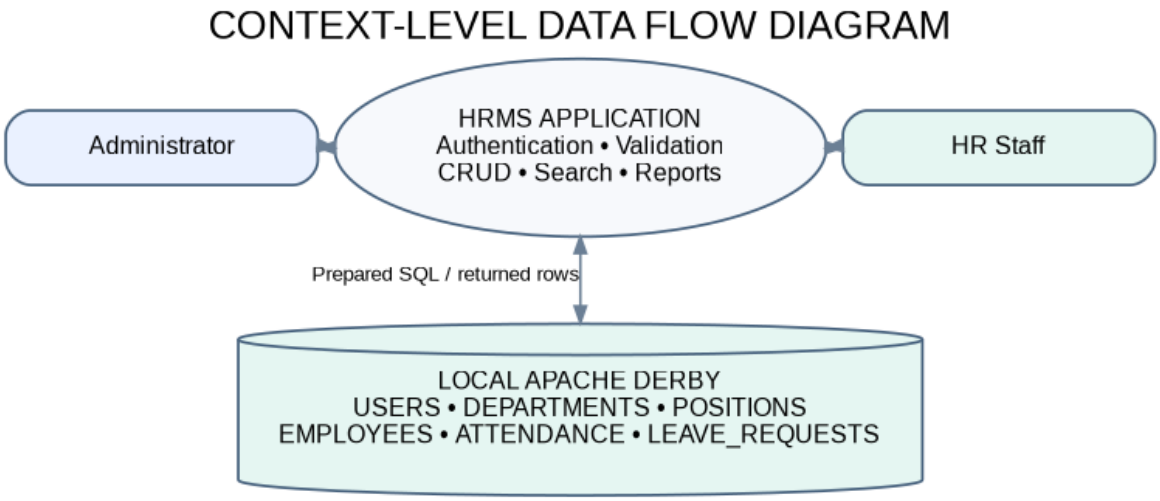


Figure 2.3. Context-level flow between users, application processing, and database storage.

2.9 Entity Relationship Diagram

Departments and positions each have a one-to-many relationship with employees. One employee may have many attendance rows and many leave requests. The users table is separated from employee records because system credentials and role controls have a different purpose.

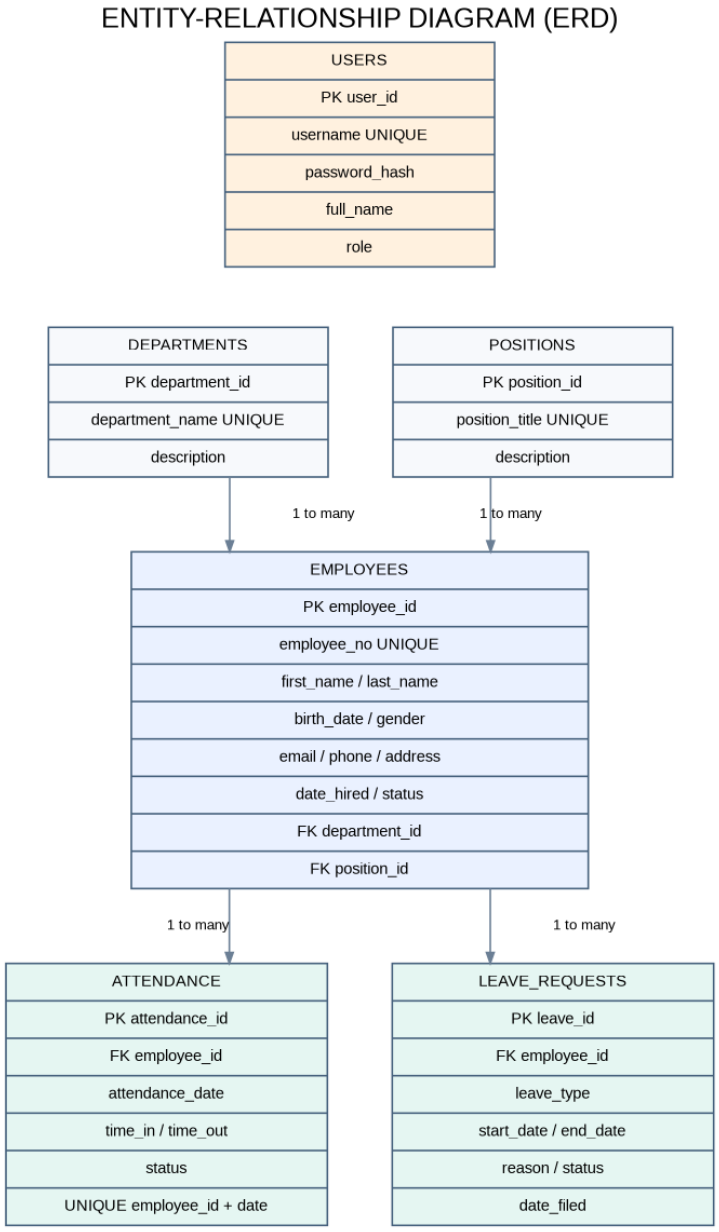


Figure 2.4. HRMS entities, key fields, and relationships.

2.10 Database Design

The schema contains six normalized tables. Auto-generated identity columns serve as primary keys, while foreign keys connect the related records. Employee numbers, usernames, department names, and position titles are unique. Attendance has a composite unique constraint on employee_id and attendance_date. The attendance schema intentionally does not require time_out to be later than time_in because overnight shifts may end on the next calendar day. Leave end dates cannot precede start dates, and check constraints limit roles and status values.

Table	Main fields and constraints	Purpose
USERS	user_id INTEGER PK IDENTITY; username VARCHAR(30) UNIQUE NOT NULL; password_hash VARCHAR(255) NOT NULL; full_name VARCHAR(80) NOT NULL; role VARCHAR(20) CHECK ADMIN/HR_STAFF.	Authentication and role-based access.
DEPARTMENTS	department_id INTEGER PK IDENTITY; department_name VARCHAR(60) UNIQUE NOT NULL; description VARCHAR(150).	Reference department names and descriptions.
POSITIONS	position_id INTEGER PK IDENTITY; position_title VARCHAR(60) UNIQUE NOT NULL; description VARCHAR(150).	Reference job titles and descriptions.
EMPLOYEES	employee_id INTEGER PK IDENTITY; employee_no VARCHAR(15) UNIQUE; first_name/last_name; birth_date DATE; gender; email/phone/address; date_hired DATE; status CHECK; department_id FK; position_id FK.	Personal, contact, and employment details linked to reference records.
ATTENDANCE	attendance_id INTEGER PK IDENTITY; employee_id FK; attendance_date DATE; time_in/time_out TIME; status CHECK; UNIQUE(employee_id, attendance_date); time-order CHECK.	One daily attendance record per employee.
LEAVE_REQUESTS	leave_id INTEGER PK IDENTITY; employee_id FK; leave_type; start_date/end_date; reason; status CHECK PENDING/APPROVED/REJECTED; date_filed DATE; CHECK(end_date >= start_date).	Leave filing and review history.

Referential integrity prevents an employee from referring to a non-existent department or position and prevents attendance/leave rows from referring to a non-existent employee. Referenced parent rows cannot be deleted while child records remain, so the user must correct or remove related records first. The database is created automatically by DatabaseSetup.java. The SQL file in the project ZIP documents the matching Derby DDL; it is a reference/manual setup script for a fresh empty database, not a phpMyAdmin import file.

CHAPTER 3 - SYSTEM DEVELOPMENT

3.1 Development Tools

Tool / Technology	Purpose
Java Development Kit 17+	Compiles and runs the application.
Apache NetBeans	IDE for opening, running, and debugging the Maven desktop project.
Java Swing	Login, dashboard, forms, tables, menus/buttons, dialogs, and report views.
Java AWT	Layouts, colors, fonts, dimensions, and event support.
JDBC	Connections, PreparedStatement, ResultSet, and SQLException handling.
Apache Derby 10.16.1.1	Embedded relational database and embedded JDBC driver.
Maven	Project/dependency management and Java compilation/execution configuration.

3.2 Java Swing

Swing provides the principal interface widgets: JFrame windows, JPanel layout containers, JLabel field labels, JTextField/JPasswordField inputs, JComboBox choices, JButton actions, JTable record/report displays, JScrollPane scrolling, JFileChooser export destinations, and JOptionPane feedback. The UI uses a light background, dark readable text, high-contrast buttons, separate page panels, and visible table headings to improve readability.

3.3 Java AWT

AWT supports the layout and visual structure through BorderLayout, GridBagLayout, GridLayout, FlowLayout, Dimension, Color, Font, and event handling. The program is event-driven: button clicks trigger authentication, validation, CRUD actions, report refresh, CSV export, or logout.

3.4 JDBC

JDBC provides the standard interface between Java and Derby. The application obtains Connection objects through a centralized utility, builds PreparedStatement objects for database operations, uses ResultSet to read query results, and handles SQLException failures. User-entered values are bound to placeholders rather than concatenated into SQL text. Dynamic SQL fragments for the generic CRUD panel come from fixed internal entity configurations, not from arbitrary user input.

3.5 Apache Derby

Apache Derby is a relational database implemented in Java. This project uses the embedded driver so the database engine runs inside the application; no external database server has to be installed or started. Derby 10.16.1.1 is used with Java 17 or later. It stores the `hrmsdb` database as a local folder relative to the program working directory by default. This configuration is suitable for a single-computer academic prototype, but it is not a multi-user client/server deployment and is not managed through phpMyAdmin.

3.6 System Architecture

The application separates presentation, validation/action handling, data access, JDBC, and persistent storage. The reusable EntityPanel renders module forms and tables based on a fixed EntityConfig. Entity-specific DAO classes extend GenericDAO; GenericDAO centralizes CRUD/search statement handling, while UserDAO provides the password-verification query. DatabaseSetup owns table creation and demo/reference seeding.

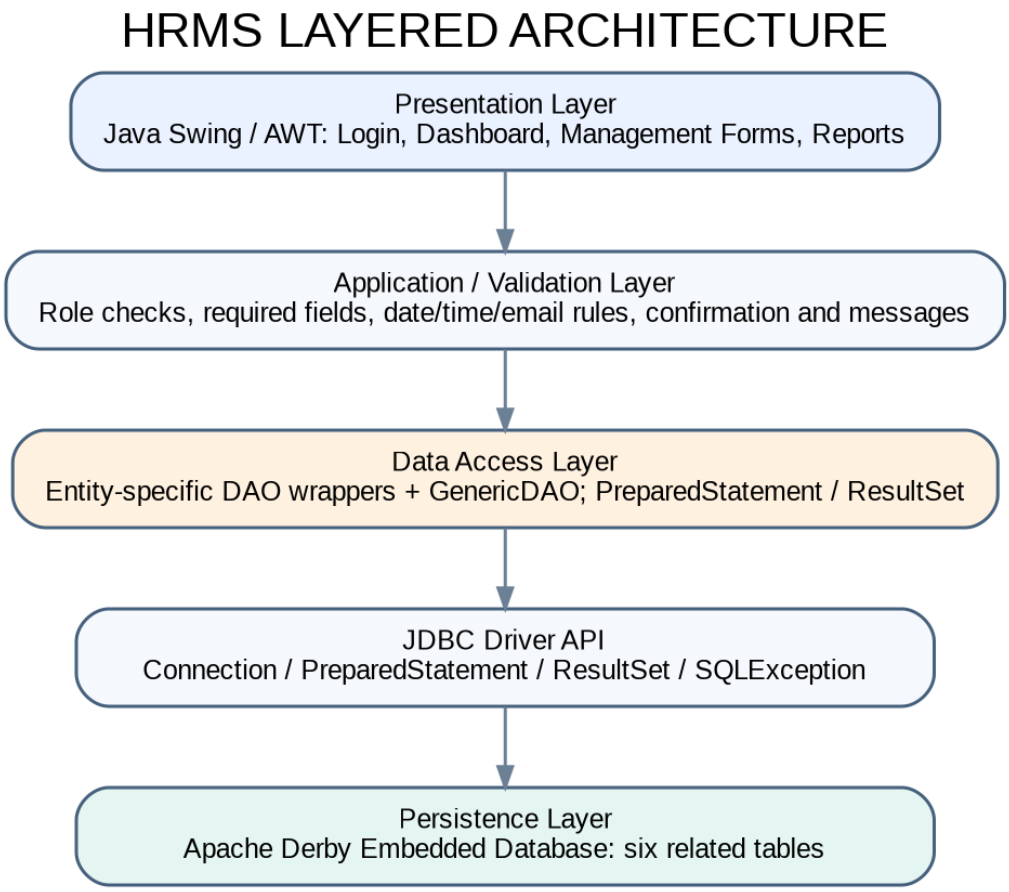


Figure 3.1. Layered architecture of the Human Resource Management System.

3.7 Program Structure

Package / file group	Examples	Responsibility
hrms	Main.java	Initializes Derby and opens the login frame.
hrms.ui	LoginFrame, MainFrame, DashboardPanel, EntityPanel, ReportsPanel, UITheme	Displays the interface, navigation, validation feedback, module forms, and reports.
hrms.ui configuration	EntityConfig, EntityConfigs	Defines fixed table/view SQL, field widgets, required fields, lookups, and searchable expressions.
hrms.model	User, Employee, Department, Position, Attendance, LeaveRequest	Encapsulates core domain data with constructors, fields, and accessors.
hrms.dao	GenericDAO, UserDAO, EmployeeDAO, DepartmentDAO, PositionDAO, AttendanceDAO, LeaveDAO	Executes database CRUD/search functions and authenticates the user.

Package / file group	Examples	Responsibility
hrms.util	DBConnection, DatabaseSetup, PasswordUtil, Validator	Centralizes connection setup, schema creation, password hashing/verification, and data validation.

3.8 Database Connection

The connection class centralizes the JDBC URL instead of repeating it across forms. The default URL is `jdbc:derby:hrmsdb;create=true`, with the working directory resolved by the utility. The database path may be overridden with the Java system property `hrms.db.path` or the `HRMS_DB_PATH` environment variable.

```
private static final String URL = "jdbc:derby:" + normalizePath(DATABASE_PATH) + ";create=true";

public static Connection getConnection() throws SQLException {
    return DriverManager.getConnection(URL);
}
```

At startup, DatabaseSetup checks Derby metadata for each required table and creates a missing table in dependency order. It then creates demonstration account hashes using PasswordUtil and inserts starter department/position records only when those names do not already exist. The schema is therefore initialized by the same program that uses it.

3.9 CRUD Implementation

Each module provides Add, Save Changes, Delete, Clear Form, Search, and Refresh actions according to the role. The EntityConfig definitions specify table names, primary-key columns, display queries, field types, and search expressions. GenericDAO builds SQL from those trusted internal definitions and binds input values with PreparedStatement. Before inserting or updating, the UI checks required fields and applicable format/date/time rules. Database constraints provide a second layer of protection.

```
String sql = "INSERT INTO departments (department_name, description) VALUES (?, ?)";
try (Connection connection = DBConnection.getConnection();
     PreparedStatement statement = connection.prepareStatement(sql)) {
    statement.setString(1, departmentName);
    statement.setString(2, description);
    statement.executeUpdate();
}
```

Search combines fixed configured expressions with a parameterized keyword pattern. Read queries join employee rows to department and position names, and join attendance/leave records to employees for readable display. Before a delete, the system requires an Administrator role and presents a confirmation dialog. Database integrity errors are converted to an understandable message where possible.

3.10 GUI and Reports

Login includes username/password, show-password, Sign In, Clear/Reset, and Exit. After authentication, the main window provides a light workspace, navy sidebar, current-user label, role-aware navigation, and sign-out. The dashboard queries employee, department, attendance, and pending-leave summaries. Module pages provide validated forms, searchable/sortable tables, and CRUD actions. Reports cover the employee directory, department headcount, attendance, leave, and pending leave requests; they are read-only and export to CSV. Administrators manage accounts, delete records, and review leave, while HR Staff cannot perform those restricted actions. The current account and last Administrator are protected.

CHAPTER 4 - TESTING

4.1 Test Plan

Testing must be conducted in NetBeans using JDK 17+ with the Maven Derby dependency resolved. Use synthetic data. The test plan covers initialization, authentication, CRUD, duplicate and foreign-key constraints, search/sort, field/date/time validation, role restrictions, reporting/export, data persistence, and logout. The Java source compiled successfully with `javac --release 17`, and a separate smoke test confirmed that password hashing produces different salted hashes and verifies correct/incorrect passwords as expected. A live Derby runtime was not available in the preparation environment; database-dependent cases remain pending until run in NetBeans.

4.2 Test Cases

Test ID	Function	Input / Action	Expected result	Runtime status
TC-01	First-run schema	Start with no `hrmsdb` database folder	Tables and demo/reference data are created.	Pending
TC-02	Administrator login	admin / admin123	Administrator dashboard and admin navigation open.	Pending
TC-03	HR Staff login	hrstaff / hr123	Dashboard opens; user-account module is hidden.	Pending
TC-04	Invalid login	Wrong password	Access denied with a useful message.	Pending
TC-05	Clear / Exit	Use Clear / Reset; then Exit	Fields clear; Exit closes after confirmation.	Pending
TC-06	CRUD / update	Create and change department, position, employee	Records persist and refreshed tables show changes.	Pending
TC-07	Required/email validation	Blank required field or malformed email	Save is rejected and validation message appears.	Pending
TC-08	Search / JOIN	Search employee by number/name	Matching row shows department and position labels.	Pending
TC-09	Delete control	Sign in as HR Staff	Delete is disabled; handler rejects unauthorized deletion.	Pending
TC-10	Foreign keys	Delete referenced department/position/employee	Database prevents invalid references and displays an error.	Pending
TC-11	Duplicate attendance	Repeat employee and attendance date	Unique constraint rejects duplicate.	Pending
TC-12	Overnight attendance	Date 2026-10-17; Time In 07:00 PM; Time Out 06:00 AM	Record saves; Time Out is displayed as 6:00 AM (next day).	Pending
TC-13	Leave date order	End date earlier than start date	Validation/database constraint rejects the row.	Pending
TC-14	Leave approval	HR Staff attempts change; Administrator updates status	Staff status remains protected; Administrator decision persists.	Pending
TC-15	User account safety	Try deleting self or last Administrator	Application prevents deletion/demotion.	Pending
TC-16	Reports and CSV	Open each report and export	Rows display in JTable; CSV contains headers and escaped values.	Pending
TC-17	Persistence/logout	Save records, sign out, close and reopen	Records persist and logout returns to login.	Pending

Test ID	Function	Input / Action	Expected result	Runtime status
TC-18	Error handling	Use invalid DB path or violate a constraint	An understandable startup/operation message is shown.	Pending

The Actual Result and Status fields above are intentionally not marked Passed because the live Derby application and complete user interface have not yet been exercised against a runtime database in this environment. The tester should record observed outcomes, screenshots/evidence, test date, and defects after completing each case.

4.3 Security and User Acceptance Testing

The source implements salted PBKDF2-HMAC-SHA256 password verification, prepared statements, field validation, role-aware navigation and action handlers, and database constraints. These measures require verification through the runtime checklist. The prototype is not yet certified for production use, and security review should include account lifecycle, backup access, sensitive-data handling, and broader deployment controls.

For user-acceptance testing, ask intended HR users to rate the statements below from 1 (Strongly Disagree) to 5 (Strongly Agree). Record the actual number of respondents, scores, average per item, overall mean, comments, and test setting. No user-acceptance scores are supplied because no completed survey was provided.

No.	UAT statement	Mean / Result
1	Labels, buttons, and navigation are easy to understand.	Pending
2	Employee details are easy to add and update.	Pending
3	Search results and joined information are useful.	Pending
4	Attendance and leave forms are straightforward to use.	Pending
5	Validation, success, and error messages are clear.	Pending
6	Reports show useful information and export correctly.	Pending
7	Administrator-only actions are properly restricted.	Pending
8	The system supports the HR tasks within its stated scope.	Pending

CHAPTER 5 - CONCLUSION

5.1 Summary

The project presents a Java desktop HRMS that centralizes core employee and HR records in an embedded Apache Derby database. The application source includes a readable Swing/AWT interface, role-aware navigation, a dashboard, management forms, JDBC CRUD/search operations, data-validation rules, foreign-key and uniqueness constraints, a user-account module, and display/export reports. The schema has six related tables and the database is initialized automatically at startup.

The implementation has been brought into alignment with the Java System Project Requirements PDF and the project paper's Derby-based architecture. It uses Java 17+ and Maven with Apache Derby 10.16.1.1. Passwords are stored using salted PBKDF2-HMAC-SHA256 rather than plain text. Code compilation and password-verification smoke testing were completed; live database functionality and user acceptance remain to be tested in the intended NetBeans environment.

5.2 Conclusions

1. A centralized relational database provides a consistent structure for employees, departments, positions, attendance, leave requests, and users.
2. Swing/AWT, JDBC, and embedded Apache Derby satisfy the technology direction required in the assignment, while the report screen addresses the general requirement to display useful reports.
3. Primary keys, foreign keys, unique constraints, check constraints, prepared statements, validation, and role-specific handlers provide multiple layers of data integrity and access control.
4. The project is a course prototype and should not be described as fully runtime-tested until the provided test matrix and user-acceptance procedure are completed.

5.3 Recommendations

1. Open the project in NetBeans with JDK 17 or newer and let Maven download the embedded Derby dependency.
2. Run every test listed in Chapter 4, record actual results, attach screenshots for important cases, and fix any failure before submission.
3. Change the demonstration credentials before using real employee data and maintain regular backups of the `hrmsdb` folder.
4. Review privacy, access, record retention, and deployment requirements before using the application outside an academic demonstration.
5. Keep the documentation and schema synchronized if any module, field, constraint, or role rule changes.

5.4 Possible Future Improvements

Improvement	Description
Leave balances	Define allowance by leave type and calculate remaining balances.
Payroll	Add salary, deductions, tax, and payslip processes after confirming local rules.
Audit logs	Record who created/updated/deleted records and when.
PDF/Excel reports	Add print layouts and formatted export options beyond CSV.
Employee self-service	Provide a secure interface for employees to review profiles and submit leave requests.
Multi-user deployment	Replace embedded Derby with a suitable server database only if the approved project scope changes.
Accessibility review	Test keyboard navigation, scaling, color contrast, and screen-reader behavior.

REFERENCES

Apache Software Foundation. (2022). Apache Derby 10.16.1.1 release notes. https://db.apache.org/derby/releases/release-10_16_1_1.cgi

Apache Software Foundation. (n.d.). Apache Derby. https://db.apache.org/derby/

Armstrong, M., & Taylor, S. (2020). Armstrong's handbook of human resource management practice (15th ed.). Kogan Page.

Bondarouk, T. V., & Ruël, H. J. M. (2009). Electronic human resource management: Challenges in the digital era. The International Journal of Human Resource Management, 20(3), 505-514. https://doi.org/10.1080/09585190802707235

Coronel, C., & Morris, S. (2019). Database systems: Design, implementation, & management (13th ed.). Cengage Learning.

Elmasri, R., & Navathe, S. (2016). Fundamentals of database systems (7th ed.). Pearson.

Horstmann, C. S. (2018). Core Java: Volume I - Fundamentals (11th ed.). Prentice Hall.

International Organization for Standardization. (2011). Systems and software engineering - Systems and software quality requirements and evaluation (SQuaRE) - System and software quality models (ISO/IEC 25010:2011).

Kendall, K. E., & Kendall, J. E. (2019). Systems analysis and design (10th ed.). Pearson.

Loy, M., Eckstein, R., Wood, D., Elliott, J., & Cole, J. (2002). Java Swing (2nd ed.). O'Reilly Media.

Oracle. (n.d.). JDBC basics. The Java Tutorials. https://docs.oracle.com/javase/tutorial/jdbc/basics/

Oracle. (n.d.). Creating a GUI with Swing. The Java Tutorials. https://docs.oracle.com/javase/tutorial/uiswing/

Pressman, R. S., & Maxim, B. R. (2020). Software engineering: A practitioner's approach (9th ed.). McGraw-Hill Education.

Strohmeier, S. (2007). Research in e-HRM: Review and implications. Human Resource Management Review, 17(1), 19-37. https://doi.org/10.1016/j.hrmr.2006.11.002

Valacich, J. S., & George, J. F. (2020). Modern systems analysis and design (9th ed.). Pearson.

APPENDIX A - RUNNING THE PROJECT

A.1 Open in NetBeans

1. Extract `HRMS_NetBeans_Project_Derby.zip` to a writable folder.
2. Install JDK 17 or newer and open Apache NetBeans with Maven support.
3. Choose File > Open Project and select the `hrms_netbeans` directory that contains `pom.xml`.
4. Allow Maven to resolve `org.apache.derby:derby:10.16.1.1` and the configured Maven plugins.
5. Run the project. The application creates `hrmsdb` and the tables automatically on first launch.
6. Use `admin` / `admin123` for Administrator or `hrstaff` / `hr123` for HR Staff during class testing.
7. After testing, change demo passwords and retain only synthetic data in coursework evidence.

A.2 Database Objects

Table	Relationship / integrity rule
USERS	Unique username; role limited to ADMIN or HR_STAFF; password_hash stores salted PBKDF2 verifier.
DEPARTMENTS	Unique department_name; referenced by employees.
POSITIONS	Unique position_title; referenced by employees.
EMPLOYEES	Unique employee_no; requires a department and position foreign key.
ATTENDANCE	References employee; only one row for each employee/date; time_out cannot precede time_in when both are present.
LEAVE_REQUESTS	References employee; end_date cannot precede start_date; status is PENDING, APPROVED, or REJECTED.

The application’s DatabaseSetup.java contains the executable schema initialization logic. The `database/hrms_schema.sql` file in the project source mirrors that schema and is provided for inspection or manual setup against a newly created, empty Derby database. Do not execute it over a database whose tables already exist.

APPENDIX B - CLASS DIAGRAM

The class/package diagram summarizes the primary interface classes, shared metadata configuration, DAO operations, utility classes, model records, and their connection to the embedded Derby database. It is an architectural overview rather than a generated UML export of every field and method in the source code.

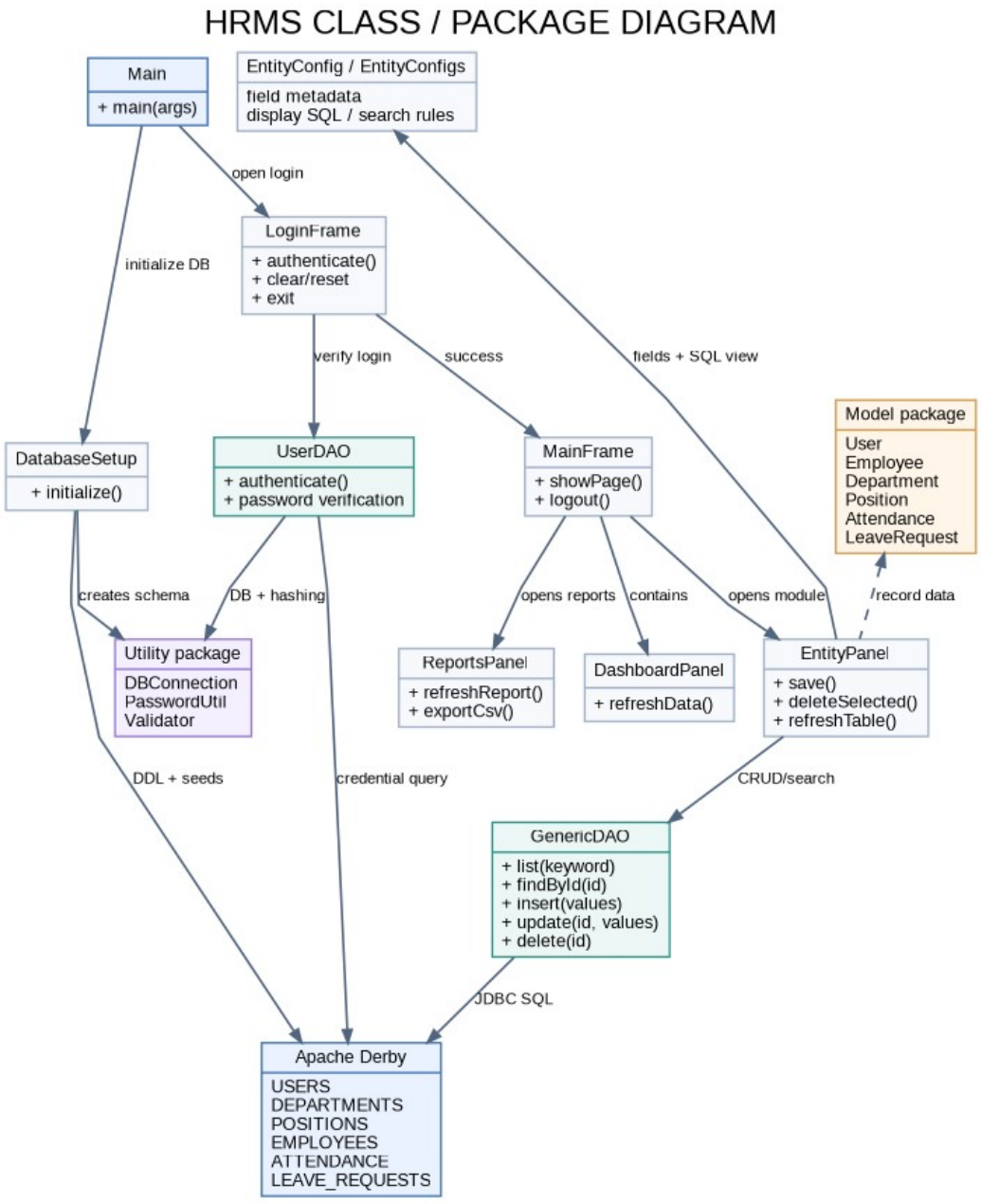


Figure B.1. HRMS class and package overview.

APPENDIX C - TABLE RELATIONSHIP DIAGRAM

Departments and Positions each relate to many Employee records. Each Employee can have multiple Attendance records and multiple Leave Requests, while the database constraint limits Attendance to one record per employee per date. Users are maintained independently for authentication and role management.

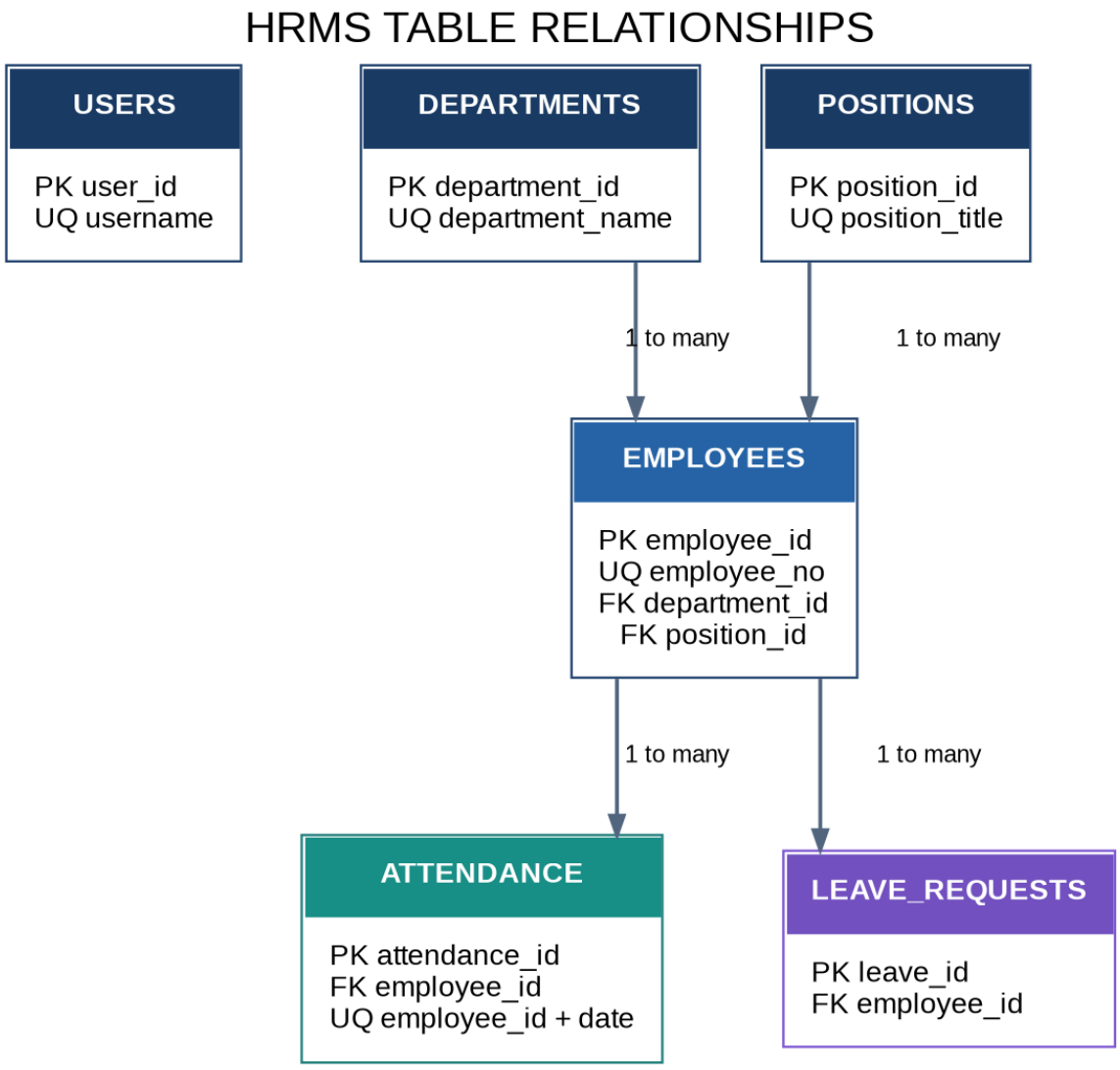


Figure C.1. Primary keys, foreign keys, and one-to-many relationships in the Derby schema.

APPENDIX D - DATABASE DICTIONARY

The dictionary lists the fields and constraints used by DatabaseSetup.java. ID columns are generated identity primary keys. Optional fields may contain NULL; required fields are marked NOT NULL.

D.1 USERS, DEPARTMENTS, and POSITIONS

Table	Field	Data type	Constraint / purpose
USERS	user_id	INTEGER	PK; generated identity.
USERS	username	VARCHAR(30)	NOT NULL; UNIQUE login name.
USERS	password_hash	VARCHAR(255)	NOT NULL; salted PBKDF2 verifier, not plain-text password.
USERS	full_name	VARCHAR(80)	NOT NULL; display name.
USERS	role	VARCHAR(20)	NOT NULL; CHECK ADMIN or HR_STAFF.
DEPARTMENTS	department_id	INTEGER	PK; generated identity.
DEPARTMENTS	department_name	VARCHAR(60)	NOT NULL; UNIQUE department name.
DEPARTMENTS	description	VARCHAR(150)	Optional description.
POSITIONS	position_id	INTEGER	PK; generated identity.
POSITIONS	position_title	VARCHAR(60)	NOT NULL; UNIQUE position title.
POSITIONS	description	VARCHAR(150)	Optional description of duties.

D.2 EMPLOYEES

Field	Data type	Constraint / purpose
employee_id	INTEGER	PK; generated identity.
employee_no	VARCHAR(15)	NOT NULL; UNIQUE office identifier.
first_name	VARCHAR(40)	NOT NULL.
last_name	VARCHAR(40)	NOT NULL.
birth_date	DATE	NOT NULL.
gender	VARCHAR(10)	NOT NULL.
email	VARCHAR(80)	Optional contact email; validated when supplied.
phone	VARCHAR(20)	Optional contact number.
address	VARCHAR(150)	Optional home address.
date_hired	DATE	NOT NULL.
status	VARCHAR(15)	NOT NULL; CHECK ACTIVE, ON LEAVE, or RESIGNED.
department_id	INTEGER	NOT NULL; FK to DEPARTMENTS.department_id.
position_id	INTEGER	NOT NULL; FK to POSITIONS.position_id.

D.3 ATTENDANCE and LEAVE_REQUESTS

Table	Field	Data type	Constraint / purpose
ATTENDANCE	attendance_id	INTEGER	PK; generated identity.
ATTENDANCE	employee_id	INTEGER	NOT NULL; FK to EMPLOYEES.employee_id.
ATTENDANCE	attendance_date	DATE	NOT NULL.
ATTENDANCE	time_in	TIME	Optional arrival time.
ATTENDANCE	time_out	TIME	Optional departure time; cannot precede time_in when both exist.
ATTENDANCE	status	VARCHAR(15)	NOT NULL; CHECK PRESENT, LATE, ABSENT, or ON LEAVE.
ATTENDANCE	employee_id + attendance_date	Composite UNIQUE	Prevents duplicate daily attendance per employee.
LEAVE_REQUESTS	leave_id	INTEGER	PK; generated identity.
LEAVE_REQUESTS	employee_id	INTEGER	NOT NULL; FK to EMPLOYEES.employee_id.
LEAVE_REQUESTS	leave_type	VARCHAR(30)	NOT NULL.
LEAVE_REQUESTS	start_date / end_date	DATE	Both NOT NULL; end_date must be on/after start_date.
LEAVE_REQUESTS	reason	VARCHAR(200)	Optional explanation.
LEAVE_REQUESTS	status	VARCHAR(15)	NOT NULL; DEFAULT PENDING; CHECK PENDING, APPROVED, REJECTED.
LEAVE_REQUESTS	date_filed	DATE	NOT NULL; date request was filed.

The schema is designed around separate entities rather than storing multiple department, position, attendance, or leave values in one employee column. Foreign keys maintain references. The user interface also validates important fields, while database constraints remain the final check for unique values and record relationships.

APPENDIX E - SQL STATEMENT EXAMPLES

The examples below use Derby SQL and PreparedStatement placeholders. In Java code, bind every ? parameter with the appropriate setString, setDate, setTime, or setInt method; do not concatenate user input into SQL statements.

E.1 Search employee records across related tables

```
SELECT e.employee_id, e.employee_no, e.first_name, e.last_name,
      d.department_name, p.position_title, e.status
FROM employees e
JOIN departments d ON d.department_id = e.department_id
JOIN positions p ON p.position_id = e.position_id
WHERE e.employee_no LIKE ? OR e.first_name LIKE ? OR e.last_name LIKE ?
ORDER BY e.last_name, e.first_name
```

E.2 Add an employee

```
INSERT INTO employees (employee_no, first_name, last_name, birth_date,
  gender, email, phone, address, date_hired, status, department_id, position_id)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
```

E.3 Record attendance and protect daily uniqueness

```
INSERT INTO attendance (employee_id, attendance_date, time_in, time_out, status)
VALUES (?, ?, ?, ?, ?)
```

The UNIQUE(employee_id, attendance_date) constraint rejects a second record for the same employee/date. Time In and Time Out are entered using a 12-hour clock with an AM or PM checkbox selection and stored as SQL TIME values. When Time Out is earlier than Time In, the system treats the clock-out as occurring on the following calendar day; attendance_date remains the date on which the shift began. The obsolete CK_ATTENDANCE_TIME constraint is removed automatically from an existing local Derby database during startup.

E.4 Department headcount report

```
SELECT d.department_name,
      COUNT(e.employee_id) AS employee_count
FROM departments d
LEFT JOIN employees e ON e.department_id = d.department_id
GROUP BY d.department_name
ORDER BY d.department_name
```

E.5 Review a leave request

```
UPDATE leave_requests
SET status = ?
WHERE leave_id = ?
```

The UI permits this status change only for an Administrator and restricts the new value to APPROVED or REJECTED. Deletion examples should only be executed after authorization and confirmation, and records referenced by foreign keys cannot be deleted until dependencies are handled.

APPENDIX F - SYSTEM INTERFACE SCREENSHOTS

The screenshots below document the intended visual style: a high-contrast light workspace, readable labels, white cards, clear table headings, and navy navigation. The login image is an actual capture of the Swing LoginFrame. The dashboard, employee-management, and reports images are illustrative interface previews using synthetic sample data; they are not evidence of a successful live database session. Capture replacement screenshots in NetBeans after completing the runtime tests in Chapter 4.

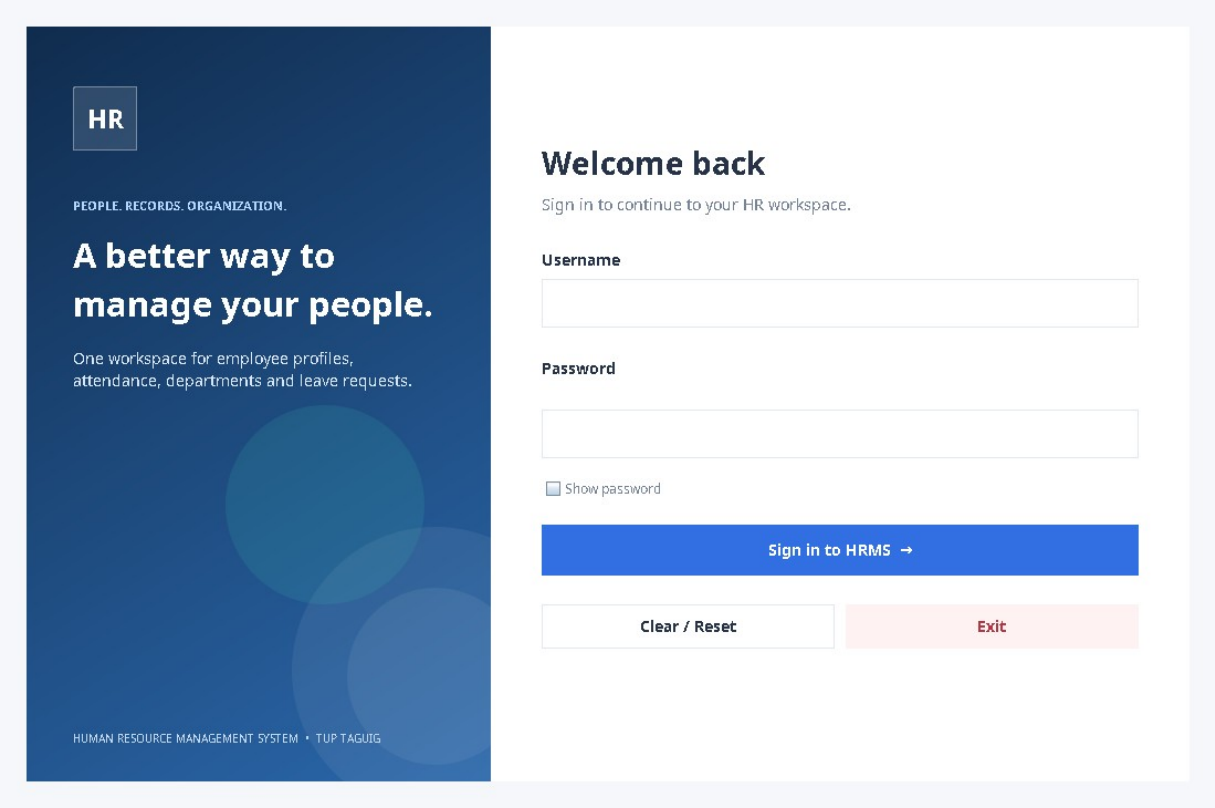


Figure F.1. Login frame, including Sign In, Clear / Reset, Exit, and password visibility controls.

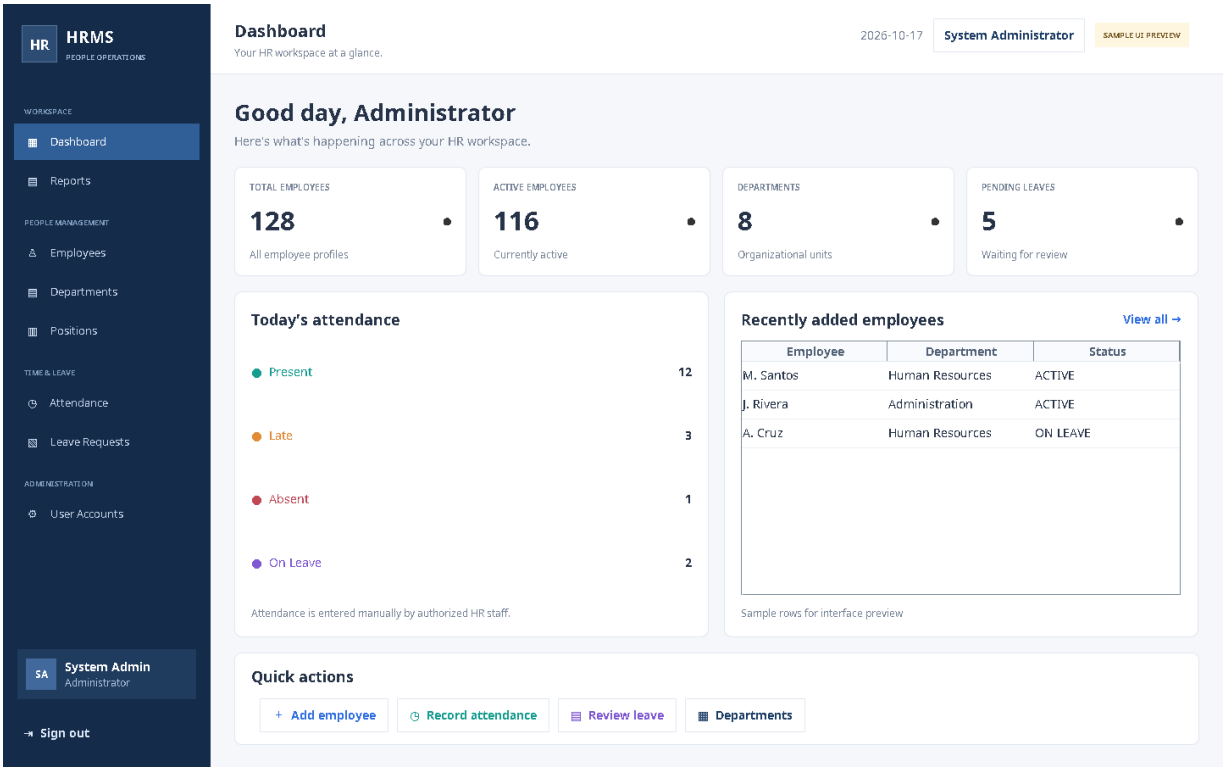


Figure F.2. Dashboard layout preview. All metric and employee rows shown are synthetic illustration data.

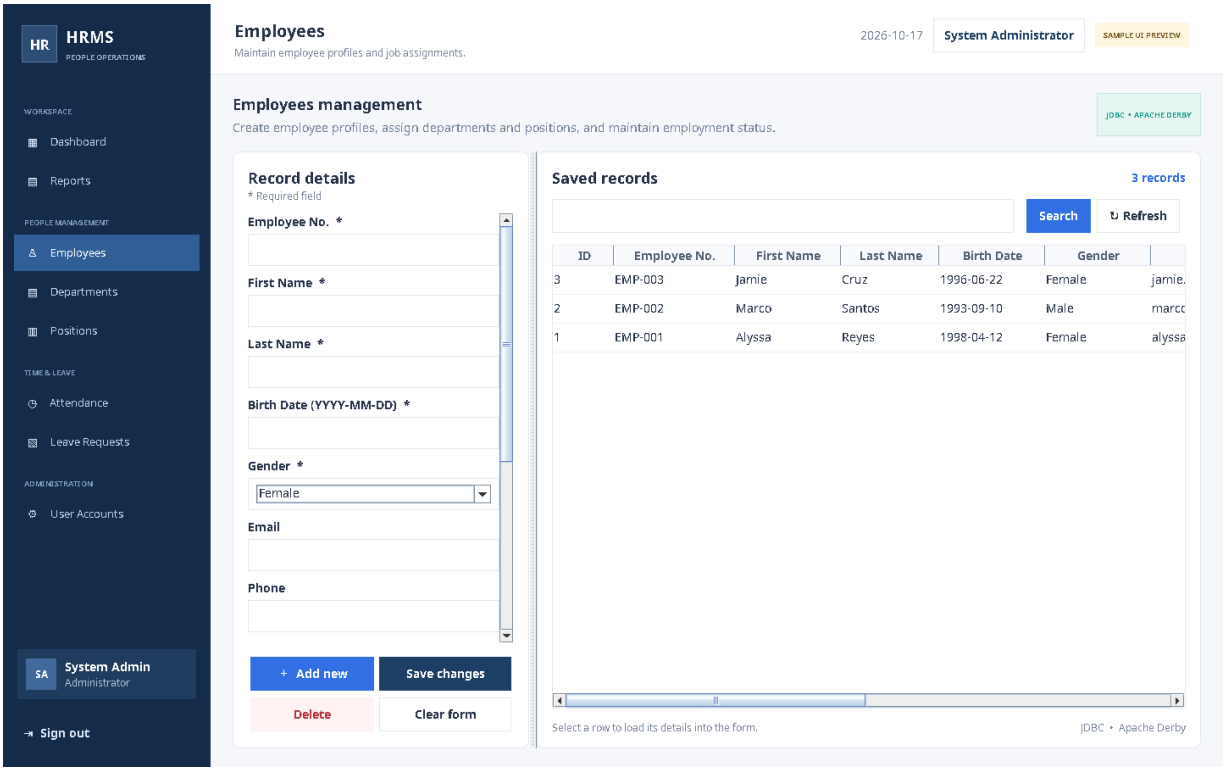


Figure F.3. Employee management form and searchable saved-record table preview using synthetic data.

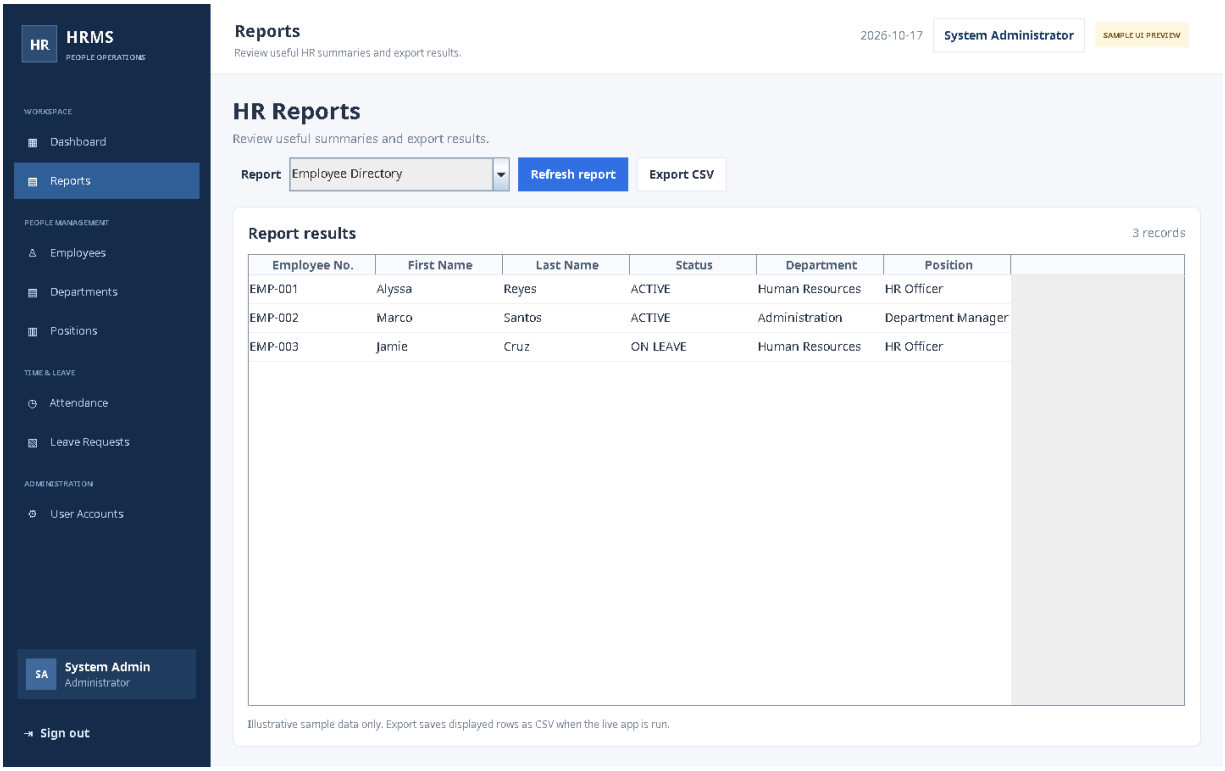


Figure F.4. Read-only report table and CSV export controls preview using synthetic data.

Runtime evidence still required: after opening the project in NetBeans and confirming that Derby initializes, replace or supplement these preview images with screenshots of actual database-backed CRUD, search, constraints, Administrator/HR Staff restrictions, report refresh/export, and logout behavior.